

Übungsblatt 9: Multicore & Finale (Klausurvorbereitung)

Mikrocomputertechnik 3 (MCT3)

Musterlösung

Zielsetzung

Sie vertiefen Vorlesung 9: Cache-Kohärenz / MESI, Race Conditions und RISC-V-Atomics (LR/SC) sowie eine kurze Klausur-Selbstkontrolle über Speicher, Interrupts, PWM und DMA.

Bearbeitungszeit: ca. 45–60 Min. | **Orientierungspunkte:** Aufgabe 1: 8 P, Aufgabe 2: 8 P, Aufgabe 3: 8 P (Gesamt: 24 P)

Teil 1: Theorie — Cache-Kohärenz und MESI

Aufgabe 1.1: Veraltete Daten im SMP

In einem SMP-SoC hat jeder Kern einen privaten L1-Cache mit *Write-Back*.

1. Beschreiben Sie das Cache-Kohärenz-Problem, wenn Core 0 eine Variable ändert und Core 1 sie kurz darauf lesen will. Warum darf Core 1 nicht einfach den gemeinsamen RAM lesen?
2. Warum löst man das Problem per Hardware (Bus Snooping) statt durch manuelles Software-Flushing vor jedem Lesevorgang?

Musterlösung Aufgabe 1.1

1. Write-Back schreibt zuerst nur in den privaten L1 — der RAM bleibt veraltet (*Stale Data*). Core 1 darf den RAM nicht als Wahrheit nehmen; die aktuelle Kopie kann im L1 von Core 0 liegen.
2. Multicore teilt sich ständig Shared-Variablen (Queues, Locks, ...). Manuelles Flushen nach jedem Write zerstört den Cache-Vorteil und überlastet den Bus. Kohärenz muss unsichtbar in Hardware laufen (Snooping / Directory).

Aufgabe 1.2: MESI-Protokoll

1. Nennen Sie die vier MESI-Zustände und ihre Bedeutung.
2. Core 0 und Core 1 haben dieselbe Line beide in **S (Shared)**. Was passiert auf dem Bus, wenn Core 0 schreiben will? Welche Zustände haben die Lines danach in Core 0 und Core 1?

Musterlösung Aufgabe 1.2

1. **M (Modified)**: verändert, weicht vom RAM ab, einzigartig. **E (Exclusive)**: sauber, einzigartig. **S (Shared)**: sauber, mehrere Kopien. **I (Invalid)**: ungültig → Miss beim Zugriff.
2. Core 0 darf nicht „heimlich“ schreiben: Invalidate-Broadcast auf dem Bus. Core 1 snoopt und setzt lokal auf **I**. Danach schreibt Core 0 und geht auf **M**. Liest Core 1 später, Miss → frische Daten typisch von Core 0.

Teil 2: Theorie — Race Conditions und Hardware-Atomics

Aufgabe 2.1: Anatomie eines Software-Fehlers

Betrachten Sie `zaehler++`; auf einer globalen Variable.

1. Übersetzen Sie den Befehl logisch in die drei RMW-Schritte (Assembler-Ebene).
2. Erklären Sie chronologisch eine Race Condition: beide Kerne inkrementieren „gleichzeitig“, das Ergebnis steigt aber nur um 1 statt um 2 (Startwert 0).

Musterlösung Aufgabe 2.1

1. `lw` (lesen) → `addi` (+1) → `sw` (zurückschreiben).
2. Beispiel: Core 0 liest 0; Core 1 liest 0; Core 0 schreibt 1; Core 1 schreibt ebenfalls 1 und überschreibt Core 0 → Speicher zeigt 1, ein Update ist verloren.

Aufgabe 2.2: RISC-V Atomics (LR/SC)

Software-Locks wie `while(lock==1); lock=1`; scheitern, weil Prüfen und Setzen selbst racy sind. RISC-V nutzt `lr.w` / `sc.w`.

1. Erklären Sie `lr.w` und `sc.w`. Welche Rolle hat der Hardware-Monitor?
2. Was passiert, wenn Core 1 `sc.w` ausführt, nachdem Core 0 die Adresse bereits verändert hat?

Musterlösung Aufgabe 2.2

1. `lr.w` liest und aktiviert einen Monitor (Reservation) auf der Adresse. `sc.w` schreibt *nur*, wenn der Monitor bestätigt, dass inzwischen kein anderer Master (Kern/DMA) die Adresse verändert hat.
2. Monitor-Prüfung schlägt fehl: Store wird nicht ausgeführt; `sc.w` liefert einen Fehlercode (typisch $\neq 0$). Software muss die Sequenz ab `lr.w` wiederholen.

Teil 3: Systemverständnis (Klausur-Wrap-Up)

Kurze Selbstkontrolle quer über das Semester.

Aufgabe 3.1: Speicher und Caches

1. Warum führt Column-Major-Zugriff auf ein 2D-Array in C zu starkem Thrashing, Row-Major aber nicht?
2. Unterschied Direct Mapped vs. 4-Way Set-Associative bei der Platzwahl eines neuen Blocks?

Musterlösung Aufgabe 3.1

1. C speichert zeilenweise (Row-Major). Zeilenweise Zugriffe nutzen räumliche Lokalität (Cache-Line). Spaltenweise Sprünge oft über große Abstände → Nachbarn unnütz, häufiger Miss / Thrashing.
2. Direct Mapped: Index → *ein* fester Slot. 4-Way: Index → Set mit 4 Ways zur Wahl → weniger Konflikt-Misses.

Aufgabe 3.2: Interrupts und Timer

1. Architektonischer Unterschied CLINT vs. PLIC? Je ein typisches Einsatzgebiet.
2. Warum darf eine ISR nicht wie eine normale C-Funktion mit `ret` enden (`__attribute__((interrupt)), mret`)?

Musterlösung Aufgabe 3.2

1. **CLINT**: kernnah, geringe Latenz — lokale Timer (`mtime/mtimecmp`), Software-/IPI-Interrupts. **PLIC**: bündelt/priorisiert externe Peripherie-IRQs (UART, SPI, GPIO, ...) und meldet typisch External Interrupt.
2. ISR unterbricht asynchron; normales `ret` restauriert Kontext/`mstatus` nicht korrekt. Das Attribut erzwingt Prolog/Epilog; Abschluss mit `mret`.

Aufgabe 3.3: DMA und Aktorik

1. Rollen von ARR und CCR bei PWM?
2. Was ist ein Interrupt Storm beim SPI-Senden großer Datenmengen per IRQ — und warum hilft DMA?

Musterlösung Aufgabe 3.3

1. ARR begrenzt die Periode/Frequenz; CCR ist die Compare-Schwelle (Duty Cycle, typisch CCR/ARR im Vorlesungsmodell).
2. Schnelles SPI verlangt Bytes im μ s-Takt → millionenfache ISRs mit Push/Pop-Overhead (*Interrupt Storm*). DMA als Bus-Master kopiert SRAM → Peripherie und entlastet die CPU.

Aufgabe 3.4: AMAT-Wiederholung

Hit Time = 1 Takt, Miss Rate = 5 %, Miss Penalty = 100 Takte.

1. Geben Sie die AMAT-Formel an und berechnen Sie die AMAT.

Musterlösung Aufgabe 3.4

$$\text{AMAT} = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty}) = 1 + (0,05 \times 100) = 1 + 5 = 6 \text{ Takte}$$